

Question 1: Hospital Shift Scheduling (Backtracking Search)

A hospital needs to assign 3 doctors (D1, D2, D3) to 3 shifts (Morning, Afternoon, Night) subject to the given constraints. Apply Backtracking Search to find a valid assignment with all intermediate steps and constraint checks, and state the final valid schedule.

Step 1: Formulate as a Constraint Satisfaction Problem (CSP)

- **Variables:** D1, D2, D3 (one variable per doctor).
- **Domain of each variable:** {M, A, N} = {Morning, Afternoon, Night}, ordered as $M < A < N$.

Constraints:

Label	Constraint	Formal form
C1	D1 cannot work Night	$D1 \neq N$
C2	D2 must work before D3	$\text{shift}(D2) < \text{shift}(D3)$
C3	Only one doctor per shift	D1, D2, D3 all different
C4	D3 cannot work Morning	$D3 \neq M$
C5	Every doctor gets exactly one shift	Each variable is assigned exactly one value

Step 2: Backtracking Search Set-up

- **Variable ordering:** $D1 \rightarrow D2 \rightarrow D3$.
- **Value ordering:** $M \rightarrow A \rightarrow N$.
- **Rule:** After every assignment, all constraints whose variables are already assigned are checked. If any fails, the value is rejected and the next value is tried. If all values fail, the search backtracks to the previous variable.

Step 3: Search Trace with Constraint Checks

Step	Assignment tried	Constraint checks	Result
1	D1 = M	C1: $M \neq N$ ✓; C3: no other assignment yet ✓	Accept, go to D2
2	D1 = M, D2 = M	C3: Morning already taken by D1 ✗	Reject, try next value
3	D1 = M, D2 = A	C3: M, A different ✓; C2: D3 not yet assigned, partial check ✓	Accept, go to D3
4	D1 = M, D2 = A, D3 = M	C4: D3 cannot work Morning ✗; C3: Morning already taken ✗	Reject, try next value
5	D1 = M, D2 = A, D3 = A	C3: Afternoon already taken by D2 ✗	Reject, try next value
6	D1 = M, D2 = A, D3 = N	C1 ✓ (D1 = M); C2: $A < N$ ✓; C3: M, A, N all different ✓; C4: $N \neq M$ ✓; C5: all three assigned ✓	All constraints satisfied: SOLUTION

Step 4: Final Valid Schedule

Shift	Doctor
Morning	D1
Afternoon	D2
Night	D3

Verification: D1 is not on Night (C1 satisfied). D2 works Afternoon and D3 works Night, so D2 works before D3 (C2). Each shift has exactly one doctor (C3). D3 is not on Morning (C4). Every doctor has exactly one shift (C5).

Step 5: Exploring the Remaining Search Space (Completeness Check)

If the search is continued after the first solution, backtracking explores the remaining branches:

- **D1 = M branch:** D2 = N gives D3 = M (violates C4) or D3 = A (violates C2, since N is not before A). Dead end.
- **D1 = A branch:** D2 = M, D3 = N satisfies all constraints. This is a second valid schedule: D1 = Afternoon, D2 = Morning, D3 = Night. The branch D2 = N again fails.
- **D1 = N branch:** Rejected immediately by C1.

Conclusion: Two valid schedules exist. The first one found by backtracking (with the chosen ordering) is D1 = Morning, D2 = Afternoon, D3 = Night. The alternative is D2 = Morning, D1 = Afternoon, D3 = Night.

Question 2: Robot Path Planning in a 5×5 Grid

A robot must move from Start (S) to Goal (G) in a 5×5 grid with obstacles. Using the given constraints and the search algorithm, show the step-by-step node expansion and priority queue updates, and determine the optimal path and cost.

Step 1: Problem Definition

The grid layout was not drawn in the question paper, so the following 5×5 grid is assumed. Cells are written as (row, column), with (0,0) at the top-left. X marks an obstacle.

	0	1	2	3	4
0	S				
1		X		X	
2		X			
3				X	
4					G

- **Start:** S = (0,0).
- **Goal:** G = (4,4).
- **Obstacles:** (1,1), (1,3), (2,1), (3,3).
- **Actions:** Up, Down, Left, Right, each with cost 1. Obstacles cannot be entered.
- **Heuristic:** $h(n) = |\text{row} - 4| + |\text{col} - 4|$ (Manhattan distance to G).
- **Priority queue ordering:** $f(n) = g(n) + h(n)$, where $g(n)$ is the path cost from S. The frontier is a priority queue ordered by lowest $f(n)$. Ties are broken by FIFO order, then by smaller $h(n)$. Neighbours are generated in the order Up, Down, Left, Right.

Note: Plain BFS uses a FIFO queue and gives the same optimal cost here, because every move costs 1. The priority queue with Manhattan distance makes the same search goal-directed, so fewer nodes are expanded.

Step 2: Heuristic Values $h(n)$

	0	1	2	3	4
0	8	7	6	5	4
1	7	X	5	X	3
2	6	X	4	3	2
3	5	4	3	X	1
4	4	3	2	1	0

Step 3: Step-by-Step Node Expansion and Priority Queue Updates

Each entry in the queue is written as cell [g, h, f]. The queue is shown in priority order (front = next node to expand).

Iter	Node expanded	Neighbours generated	Priority queue after update
0	Initial	Insert S	(0,0) [0,8,8]
1	(0,0)	Down (1,0) [1,7,8]; Right (0,1) [1,7,8]. Up and Left are outside the grid.	(1,0) [1,7,8], (0,1) [1,7,8]
2	(1,0)	Down (2,0) [2,6,8]. Up (0,0) is closed; Right (1,1) is an obstacle.	(2,0) [2,6,8], (0,1) [1,7,8]
3	(2,0)	Down (3,0) [3,5,8]. Right (2,1) is an obstacle.	(3,0) [3,5,8], (0,1) [1,7,8]
4	(3,0)	Down (4,0) [4,4,8]; Right (3,1) [4,5,9].	(4,0) [4,4,8], (0,1) [1,7,8], (3,1) [4,5,9]
5	(4,0)	Right (4,1) [5,3,8]. Up (3,0) is closed.	(4,1) [5,3,8], (0,1) [1,7,8], (3,1) [4,5,9]
6	(4,1)	Right (4,2) [6,2,8]. Up (3,1) reached with g = 6, which is worse than the existing g = 4, so it is ignored.	(4,2) [6,2,8], (0,1) [1,7,8], (3,1) [4,5,9]
7	(4,2)	Right (4,3) [7,1,8]; Up (3,2) [7,3,10].	(4,3) [7,1,8], (0,1) [1,7,8], (3,1) [4,5,9], (3,2) [7,3,10]
8	(4,3)	Right (4,4) [8,0,8]. Up (3,3) is an obstacle.	(4,4) [8,0,8], (0,1) [1,7,8], (3,1) [4,5,9], (3,2) [7,3,10]
9	(4,4)	Goal node removed from the front of the queue: goal test satisfied. Search stops.	—

Step 4: Optimal Path and Cost

Tracing back the parent pointers from G:

(0,0) → (1,0) → (2,0) → (3,0) → (4,0) → (4,1) → (4,2) → (4,3) → (4,4)

Moves: Down, Down, Down, Down, Right, Right, Right, Right (8 moves).

Optimal path cost = $8 \times 1 = 8$.

Step 5: Why the Path is Optimal

- The Manhattan distance is admissible and consistent here: every move costs 1 and moves change the Manhattan distance by at most 1, so $h(n)$ never overestimates the true cost.
- The lowest possible cost from S to G is $h(S) = 8$. The path found has cost exactly 8, so no cheaper path can exist.
- The search expanded only 8 nodes before reaching G, and all of them had $f = 8$. Nodes with larger f , such as (3,1) and (3,2), were never expanded.

Question 3: Autonomous Rescue Robot in a Disaster Building

An autonomous rescue robot operates in a partially observable grid of rooms. It starts at S and must reach a survivor at G with limited sensing, blocked cells, and risky zones, minimizing total cost while navigating safely and updating its knowledge online.

Problem Understanding

- **Environment:** a grid of rooms; S = start, G = survivor location.
- **Actions:** Up, Down, Left, Right.
- **Step cost:** a normal move costs 1; entering a risky zone costs $1 + 2 = 3$.
- **Goal:** reach G with minimum total cost, never entering a blocked cell.
- **Information:** the robot senses only adjacent cells, so the map is discovered while moving.

a) Analysis of Constraints and Their Effect on Decision-Making

Constraint	Effect on the robot's decision-making
Moves: up, down, left, right	The action set is small (maximum 4 per state), so the search tree has a branching factor of at most 4. Diagonal shortcuts are not available.
Blocked cells cannot be traversed	A hard constraint. Any action leading to a blocked cell is illegal and must be removed from the action set. A newly discovered obstacle can invalidate the current plan and forces replanning.
Limited sensing (adjacent cells only)	The environment is partially observable. The robot cannot plan a complete path in advance. It must act on the known part of the map, treat unknown cells as unexplored, and plan incrementally.
Step cost 1, risky zone adds 2	Costs are non-uniform (1 or 3). Plain BFS (fewest steps) is no longer optimal. The robot may take a longer route through safe cells if it is cheaper than a shorter route through risky cells.
Minimize total cost with safe navigation	This is an optimization objective together with a safety requirement. The robot should prefer the lowest-cost route and avoid risk unless the detour costs more than the risk penalty.
Dynamic knowledge update (online search)	The robot interleaves planning and acting. After each move it updates its internal map (obstacles, risky zones, free cells) and recomputes the best path.

Prioritization of constraints: (1) Blocked cells and safety are hard constraints and are never violated. (2) Partial observability determines how the robot searches (online, with replanning). (3) Cost minimization is the optimization objective applied among all feasible actions.

b) Solution Approach: Online Uniform Cost Search with Replanning

The chosen strategy is Uniform Cost Search (UCS) used inside an online search agent. UCS always expands the node with the lowest path cost $g(n)$, so it handles the unequal step costs (1 and 3) correctly. The online agent repeats a sense, update, plan, act loop.

1. **Initialize:** Place S in the known map. Set current position = S. Mark all other cells as unknown. Assume unknown cells are free with cost 1, so the robot can plan optimistically.
2. **Sense:** Read the four adjacent cells. For each one, record whether it is free, blocked, or risky.
3. **Update knowledge:** Add the new information to the knowledge base (map memory). Remove blocked cells from the available actions.
4. **Plan:** Run UCS on the known map from the current position to G. Use the step cost $c = 1$ for a normal cell and $c = 3$ for a risky cell. The frontier is a priority queue ordered by $g(n)$. Expand the cheapest node, generate its legal neighbours, and update $g(n)$ if a cheaper route to a cell is found.
5. **Act:** Execute only the first move of the cheapest plan, and move to that cell. Add its cost to the total cost so far.
6. **Replan when needed:** If the robot finds a blocked cell or a risky zone that changes the cost of the current plan, discard the old plan and return to Step 2 (replan from the new position).
7. **Terminate:** If the current position = G, stop and begin rescue. If UCS finds no route in the known map, explore the cheapest frontier cell to discover more of the map. If the entire reachable region is explored and G is unreachable, report failure safely.

Illustration of cost-based choice: suppose a route through one risky cell and two normal cells costs $3 + 1 + 1 = 5$, while a safe detour of four normal moves costs 4. UCS selects the detour, because $4 < 5$. If the safe detour instead cost more than 5, UCS would choose the route through the risky cell.

c) Application of AI Concepts

(i) Intelligent Agent

The rescue robot is a model-based, goal-based and utility-based agent. It senses the environment through sensors and acts through actuators, and it keeps an internal model (map memory) to compensate for partial observability.

PEAS	Description
Performance measure	Minimum total path cost, reaching the survivor, safety (no collision with blocked cells, minimum time in risky zones)
Environment	Grid of rooms with obstacles, risky zones, and a survivor at G
Actuators	Wheels or tracks for movement: Up, Down, Left, Right
Sensors	Adjacent-cell sensors (obstacle detection, risk detection), position sensor

(ii) Rationality

A rational agent selects the action that is expected to maximize its performance measure, given its percept history and built-in knowledge. Here, the robot is rational because it:

- chooses the action leading to the lowest total path cost based on everything it has sensed so far (UCS on the known map);
- avoids illegal and unsafe actions (blocked cells);
- accepts a risky zone only when the extra cost of 2 is smaller than the cost of the alternative detour;
- learns from percepts by updating its map and replanning, rather than following an outdated plan.

Rationality here is about making the best decision with available information. It does not require the robot to know the whole map in advance.

(iii) Environment Type

Property	Type	Reason
Observability	Partially observable	Only adjacent cells can be sensed
Agents	Single agent	Only the robot acts (survivors are static goals)
Determinism	Deterministic (assumed)	A chosen move leads to the intended cell; the outcome is known
Episodic / Sequential	Sequential	Each move affects later moves and the total cost
Static / Dynamic	Static (assumed)	The building does not change while the robot decides; its map knowledge changes
Discrete / Continuous	Discrete	Grid cells and a finite set of actions
Known / Unknown	Unknown (initially)	The robot does not know the layout beforehand and learns it by exploring

d) Justification of the Chosen Approach

- **Handles non-uniform costs:** Because step costs are 1 or 3, the cheapest path is not the one with the fewest steps. UCS is optimal for any non-negative step cost, whereas BFS is optimal only when all costs are equal.
- **Fits partial observability:** The robot does not know the full map. An offline method, which needs the complete map before moving, would fail. The online agent plans on the known map, acts, senses, and replans.
- **Guarantees safe navigation:** UCS only returns paths through cells known to be free, and blocked cells are excluded from the action set, so the plan is always safe.
- **Cost-optimal and complete:** Because UCS always expands the lowest-cost node first, the plan it returns on the current map is cost-optimal. When the map changes, replanning keeps the route optimal for the updated knowledge.
- **Comparison with other strategies:** DFS is neither optimal nor cost-aware and may enter risky zones. BFS ignores costs. Greedy search ignores the cost already paid and

can be misled by obstacles. Hence UCS in an online loop is the most suitable of the uninformed strategies for CO1. If the survivor's location is known, adding the Manhattan distance heuristic (Online A*) reduces the number of nodes expanded without losing optimality, since the heuristic never overestimates a step cost of at least 1.

Conclusion: Online Uniform Cost Search with dynamic map updating and replanning is the most suitable approach, because it satisfies every constraint: it respects obstacles, handles unequal and risky costs, works with limited sensing, and yields a minimum-cost, safe route to the survivor.